Chapter 4

Machine Learning Fundamentals

Building Intelligent Systems with Data

Course: Data Science Application

Course Code: `InSy3056`

Learning Objectives

By the end of this chapter, you will be able to:

- Distinguish between supervised and unsupervised learning approaches
- Implement various supervised learning algorithms
- Apply cross-validation techniques for model evaluation
- Build regression and classification models
- Understand and implement unsupervised learning algorithms
- Evaluate and validate machine learning models
- Perform feature selection and engineering

4.1 Overview of Supervised and Unsupervised Learning

Machine learning is a field of artificial intelligence that focuses on building systems that can automatically learn from data and improve their performance over time, without being explicitly programmed for specific tasks.

- Two core types of machine learning are **supervised learning** and **unsupervised learning**.

What is Supervised Learning?

In supervised learning, the algorithm is trained using a labeled dataset.

- This means that for each example in the training set, the input data is paired with the correct output (label).

> The goal is for the model to learn the mapping between the input features and the target output, so it can make accurate predictions on new, unseen data.

Training Process in supervised learning

- Algorithm learns from examples with known correct answers
- Model adjusts parameters to minimize prediction errors and Performance measured against known ground truth

Types of supervised learning

- **Regression:** Predicts continuous numerical values
- **Classification:** Predicts discrete categories or classes

Key Requirements

- Large amount of labeled training data with Quality training examples
- Appropriate evaluation metrics

What is Unsupervised Learning?

Unsupervised learning deals with data that does not have explicit labels.

> The objective is to find patterns, structures, or relationships within the data without prior knowledge of the outcomes.

Training Process of unsupervised learning

- Algorithm finds patterns without guidance and no correct answers to learn from
- Discovers hidden structures in data

Types unsupervised learning

- **Clustering:** Groups similar data points
- **Association:** Finds relationships between variables
- **Dimensionality Reduction:** Simplifies data while preserving information

Key Challenges unsupervised learning

- Difficult to evaluate results objectively
- Requires domain expertise to interpret findings and more exploratory in nature

Key Differences

- **Data:** Supervised learning uses labeled data, while unsupervised learning uses unlabeled data.
- **Goal:** Supervised learning predicts outcomes or classifications; unsupervised learning discovers patterns or groupings in data.
- **Examples:** Spam detection (supervised), market segmentation (unsupervised).

Understanding these fundamental learning approaches is essential since they form the basis for most machine learning models and applications.

4.2 Supervised Learning Algorithms

Supervised learning algorithms learn from labeled examples to make predictions on new, unseen data.

- Supervised learning algorithms are the foundation of most predictive analytics and AI systems.

> Their primary goal is to learn a mapping from input features (X) to an output or target variable (y) based on example data provided to them ("labeled data"). This process enables them to predict outcomes on new, unseen data.

There are many different supervised learning algorithms, each suited to particular types of tasks and datasets. Some of the most common and important algorithms include:

- **Linear Regression:-** Used for predicting continuous numeric values.
- **Logistic Regression:-** Despite its name, this is used for classification problems (not regression).
- **Decision Trees:-** These are flexible models that can be used for both classification and regression.
- **Random Forests:-** An ensemble method that builds multiple decision trees on random subsets of the data and averages their predictions.

- **Support Vector Machines (SVM):** These create a hyperplane (or set of hyperplanes) in a high-dimensional space to separate classes.
- **k-Nearest Neighbors (k-NN):** A simple, non-parametric approach which classifies new instances by a majority vote among the k closest data points in the training set.
- **Naive Bayes:** A probabilistic classifier based on applying Bayes' theorem, assuming that features are independent given the class.
- **Neural Networks:** Highly flexible models inspired by biological neurons, capable of learning complex, non-linear relationships.

Each algorithm has its strengths and weaknesses depending on the problem type, data structure, noise in the data, and other requirements.

Choosing the right algorithm often depends on:

- The nature of the task (regression vs. classification)
- The size and dimensionality of the data
- Feature relationships and interactions
- The need for interpretability vs. predictive power
- Computational resources available

After training a supervised learning algorithm, it's crucial to evaluate its performance using appropriate metrics:

- accuracy, precision, recall, F1-score for classification
- mean squared error and others for regression
- cross-validation to ensure that the model generalizes well to unseen data.

Key Components of SL algorithms

- **Training Set:** Data used to train the model
- **Features (X):** Input variables or attributes
- **Target (y):** Output variable we want to predict
- **Model:** Mathematical function that maps inputs to outputs
- **Parameters:** Values learned during training

Classification

Classification is a supervised machine learning method where the model tries to predict the correct label of a given input data.

- Classification algorithms learn from labeled training data where each input is associated with a known class label to build a model that can assign correct class labels to new, unseen data.

There are many types of classification tasks, including:

- **Binary classification:** Where there are only two possible classes (e.g., spam or not spam).
- **Multi-class classification:** Involves more than two classes (e.g., classifying types of fruits: apple, banana, orange).
- **Multi-label classification:** Each instance can be assigned to multiple classes at the same time.

> **Classification algorithms** include `logistic regression`, `decision trees`, `support vector machines`, `k-nearest neighbors`, and `neural networks`.

> **Classification** performance is evaluated using metrics such as `accuracy`, `precision`, `recall`, `F1-score`, and the `confusion matrix`, which help measure how well the model is assigning the correct classes.

Eager vs. Lazy Learners

Eager Learners

- Eager learners build a generalized model by learning patterns from the entire training dataset up front, during the training phase.
- They process all the data and construct an internal representation (a model) before they can make predictions.
    - Decision Trees, Logistic Regression, Neural Networks, Support Vector Machines (SVM).
- Once trained, predictions are fast since the model is already built.
- Can be slow to train, may require retraining from scratch if new data arrives.

Lazy Learners

- Lazy learners do not build a model during the training phase. Instead, they store the training data and wait until a query (prediction request) is made.
- When asked to predict for new data, they perform computations (like searching or averaging) using the stored training data.
- k-Nearest Neighbors (k-NN), Case-Based Reasoning.
- Fast to adapt to new data since no retraining is needed, often simple to implement.
- Predictions can be slow, especially on large datasets, since the algorithm must compute distances or similarities at prediction time.

Types of Classification Algorithms

- **Logistic Regression:-** Estimates probabilities of class membership using the logistic (sigmoid) function.
- **Support Vector Machines (SVM):-** Finds the optimal hyperplane that best separates classes with the maximum margin.
- **Decision Trees:** Splits data into classes based on feature values in a hierarchical, tree-like structure.
- **k-Nearest Neighbors (k-NN):-** Classifies new data points based on the majority label among their k closest neighbors.
- **Naive Bayes:-** Estimates class probabilities based on Bayes' theorem, assuming strong independence between features.
- **Neural Networks:-** Learns complex, non-linear patterns through multiple layers of interconnected artificial neurons.

Ensemble Methods

Ensemble methods combine predictions from multiple models to improve overall performance, robustness, and generalization.

- **Random Forest:-** Builds an ensemble of many decision trees trained on random subsets of the data and features, then aggregates their predictions (often via majority vote for classification).
- **Gradient Boosting Machines (e.g., XGBoost, LightGBM, CatBoost):-** Sequentially build an ensemble of weak learners (usually decision trees), where each new model corrects errors made by the previous ones.
- **AdaBoost:-** Trains a sequence of weak learners, focusing on instances that previous learners misclassified, and combines them into a strong composite classifier.

Ensemble Methods

- **Bagging (Bootstrap Aggregating):-** Trains multiple models (often decision trees) on random bootstrap samples of the data and combines their predictions to reduce variance.
- **Stacking:-** Integrates multiple different types of models and learns how best to combine their outputs using a meta-model.

These algorithms and techniques can be applied to binary, multiclass, or multi-label classification tasks, depending on the specific problem and data at hand.

Logistic Regression

Logistic Regression is a fundamental algorithm used for **binary classification** tasks, where the objective is to predict one of two possible outcomes (e.g., yes/no, spam/not spam, disease/no disease).

- Logistic regression predicts the probability that an input belongs to the positive class using a linear combination of input features, passed through a *sigmoid* (logistic) function.
- The output is always between 0 and 1, representing the predicted probability.

- **Sigmoid Function**
  - The sigmoid (logistic) function transforms any real-valued number into a value between 0 and 1.
  - Formula:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

where (z = w^T x + b) (weighted sum of features plus bias).

- **Decision Boundary and Threshold**
  - By default, inputs with predicted probability greater than 0.5 are classified as class 1 (positive class), otherwise class 0.

Considerations for Logistic Regression Model Development

When developing and deploying a logistic regression model, it is important to pay attention to the following considerations:

- **Labeled Data:-** Each example in the dataset must have both input features and a known target class label (e.g., 0/1 for binary classification).
- **Numerical or Categorical Features:-** Features can be numerical (e.g., age, income) or categorical (e.g., gender, geographic region). Categorical features should be converted to numerical representation (such as one-hot encoding) before training.
- **Feature Scaling:-** Ensure features are on similar scales (e.g., using standardization or normalization), as this can impact convergence and model performance.

- **Multicollinearity:-** Logistic regression can be sensitive to correlated features. Consider removing highly correlated features or applying dimensionality reduction.
- **Irrelevant Features:-** Including irrelevant or noisy features can decrease predictive performance perform feature selection or engineering as needed.
- **Data Balance:-** If classes are imbalanced, the model may be biased toward the majority class. Use techniques such as class weighting, resampling, or collecting more data if possible.

- **Regularization:-** Apply regularization (L1, L2 penalty) to prevent overfitting, especially when there are many features relative to samples.
- **Outliers:-** Outliers can strongly influence the model. Investigate and address outliers via screening, robust scaling, or transformation if necessary.
- **Interpretability:-** Logistic regression models are interpretable; coefficients indicate the direction and magnitude of feature effects. However, interpretability depends on the absence of strong multicollinearity.

- **Sample Size:** Logistic regression generally performs better with more data, especially when the number of features is large.
- **Evaluation Metrics:** Choose appropriate metrics (e.g., accuracy, precision, recall, AUC) based on the problem, especially for imbalanced datasets.

Carefully considering these factors can lead to more reliable, generalizable, and interpretable logistic regression models.

Advantages

- Simple and easy to interpret.
- Provides probabilities, not just class labels.
- Works well for linearly separable classes.

Limitations

- Assumes a linear relationship (in the log-odds) between features and outcome.
- Not well-suited for complex, non-linear problems without feature engineering.

Basic Parameters for `sklearn` LogisticRegression

When using scikit-learn's `LogisticRegression`, some of the most important parameters you can set include:

- **`penalty`** Regularization type ('l1', 'l2', 'elasticnet', or 'none'). Default is `'l2'`.
- **`C`** Inverse of regularization strength. Smaller values specify stronger regularization. Default is `1.0`.
- **`solver`** Algorithm to use for optimization
  - `liblinear` for small data set, binary class.
  - `lbfgs` for medium to large datasets, full multi class.
  - `saga` very large dataset full multi class. Default is `'lbfgs'`.

- `max_iter` :- Maximum number of iterations for the solver. Default is `100` .
- `class_weight` :- Handles imbalanced classes. Options include `None` (default) or `'balanced'` .
- `random_state` :- Seed for reproducibility of results.
- `fit_intercept` :- Whether to include an intercept (bias) term. Default is `True` .
- `multi_class` :- Type of multiclass strategy.
  - `'auto'` - Automatically selects based on solver and problem type. Default behavior.
  - `'ovr'` - This strategy trains a separate binary classifier for each class.
  - `'multinomial'` - This strategy trains a single model that directly predicts the probabilities of all classes simultaneously.

Example usage:

```
from sklearn.linear_model import LogisticRegression

# Basic logistic regression with common parameters
model = LogisticRegression(
    penalty='l2',
    C=1.0,
    solver='lbfgs',
    max_iter=100,
    class_weight='balanced',
    random_state=42
)
model.fit(X_train, y_train)
```

Refer to the scikit-learn documentation for a full list of parameters and options.

Decision Tree Classifier

A **Decision Tree** is a non-parametric supervised learning algorithm used for both classification and regression tasks.

- A powerful tool in machine learning due to their simplicity and interpretability, and they serve as the foundation for more complex ensemble methods.

Key Characteristics

- Models data by recursively splitting it based on feature values to form a tree-like structure.
- Each internal node represents a "test" on a feature, each branch represents the outcome, and each leaf node represents a prediction.
- Handles both numerical and categorical data.

How Does a Decision Tree Work?

A decision tree in machine learning operates through a series of systematic steps to learn patterns in data and make predictions:

1. **Choosing the Root Node**
    - The algorithm examines all features in the dataset and selects the one that provides the highest information gain (or, equivalently, the lowest impurity).
    - Metrics such as **Information gain**, **Gini Index** and **Entropy** are commonly used for this evaluation.
    - The selected feature becomes the root node of the tree.

2. **Recursive Splitting**

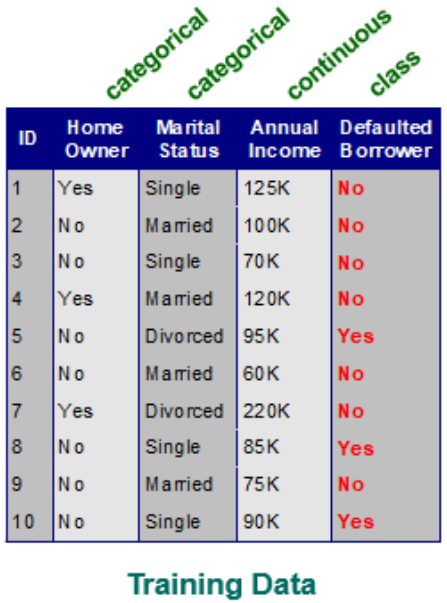
- The process repeats at each child node: the algorithm looks at the available features and determines the best one to split on (again, maximizing gain or minimizing impurity).
- At every step, the dataset is partitioned accordingly.

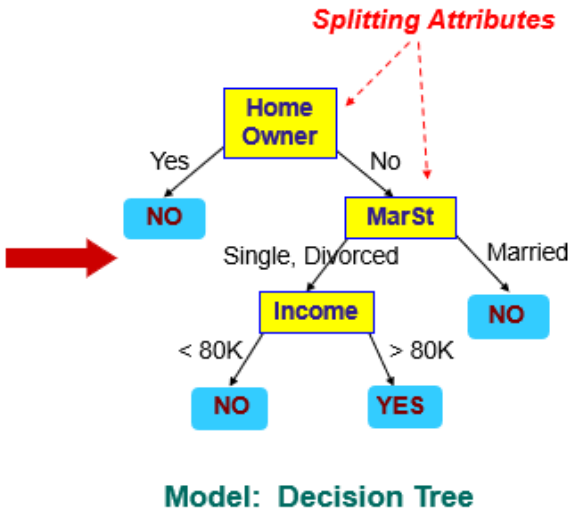
3. **Stopping Criteria**

- The recursive splitting ends when any of the following occur:
  - **Pure node**: All samples at the node belong to the same class.
  - **Maximum depth**: The tree reaches a user-defined maximum depth.
  - **Minimum samples**: The number of data points at a node becomes too small to allow further splitting.

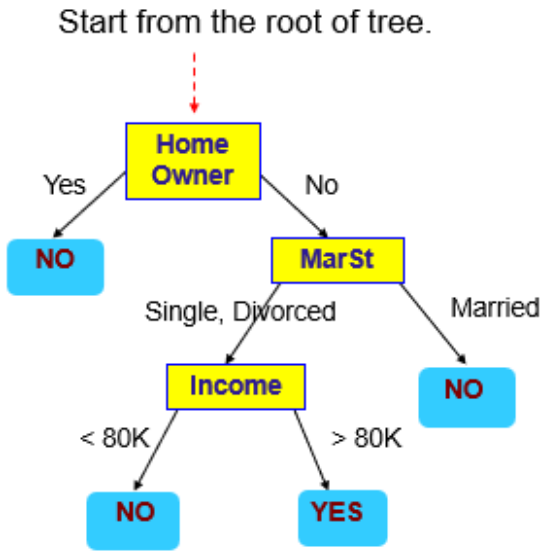
4. **Prediction**
    - With a trained tree, predictions are made by passing new input data from the root down the tree, following the decision rules at each node.
    - When a leaf node is reached, its class or value is output as the final prediction.

This step-by-step structure allows the decision tree to handle both numerical and categorical data, and provides a clear, interpretable model of decision logic.





Training Data



Test Data

Home Owner	Marital Status	Annual Income	Defaulted Borrower
No	Married	80K	?

Model: Decision Tree

Attribute Selection Measures In Decision Tree

As with any machine learning algorithm, the most important step in a decision tree algorithm is attribute selection. The common measures used to select attributes are:

Gini Index

- The Gini Index measures impurity in a dataset.
- It ranges from 0 (perfectly pure, all data belongs to one class) to 1 (completely impure).
- The algorithm selects the split that minimizes the Gini Index, ensuring each subset is as pure as possible.

$$\text{Gini}(D) = 1 - \sum_{i=1}^{C} p_i^2$$

where (pi) is the proportion of instances of class (i) in dataset (D), and (C) is the number of classes.

Gini Gain

- **Gini Gain** is another metric derived from the Gini Index, used to determine the effectiveness of an attribute in classifying the training data.
- It measures the decrease in Gini impurity caused by a split—the higher the Gini Gain, the better the attribute at separating classes.

$$\text{Gini Gain} = \text{Gini}(\text{parent}) - \sum_{j=1}^{k} \frac{|D_j|}{|D|} \text{Gini}(D_j)$$

```
Home Owner = Yes: 3 records, ALL "No" → Gini = 0.0
Home Owner = No: 7 records (4 No, 3 Yes) → Gini = 0.49
Weighted Gini = (3/10)×0.0 + (7/10)×0.49 = 0.343
Gini Gain = 0.42 - 0.343 = 0.077
```

Information Gain

- Entropy measures the randomness or disorder in the data, with lower entropy indicating higher purity.
- Based on the concept of entropy, Information Gain quantifies the reduction in uncertainty after splitting.
- A split with higher Information Gain is preferred.

$$\text{Entropy}(D) = -\sum_{i=1}^{C} p_i \log_2 p_i$$

$$\text{Information Gain} = \text{Entropy(parent)} - (\text{weighted sum of Entropy of splits})$$

Gain Ratio

- While Information Gain works well, it can favor attributes with many unique values.
- Gain Ratio normalizes Information Gain by considering the intrinsic information of a split, making it more robust for datasets with categorical features.

$$\text{Gain Ratio} = \frac{\text{Information Gain}}{\text{Split Information}}$$

where Split Information measures the potential information generated by splitting the dataset.

When to Use Each Attribute Selection Method

- For most general purposes and mixed datasets, **Gini Index** (fast and robust) is preferred.
- For datasets where interpretability in terms of information theory is important, or for categorical splits, **Information Gain** may be chosen.
- For categorical features with many unique categories, use **Gain Ratio** to reduce bias towards such attributes and achieve more meaningful splits.

Basic Parameters for `sklearn` DecisionTreeClassifier

- **`criterion`** : Function to measure the quality of a split (`'gini'` for Gini impurity, `'entropy'` for information gain). Default is `'gini'` .
- **`max_depth`** : Maximum depth of the tree. If `None` , nodes are expanded until all leaves are pure or contain less than `min_samples_split` samples.
- **`min_samples_split`** : Minimum number of samples required to split an internal node. Default is `2` .
- **`min_samples_leaf`** : Minimum number of samples that must be present in a leaf node. Default is `1` .
- **`max_features`** : Number/features considered when looking for the best split.
- **`random_state`** : Seed for reproducibility.

Example usage:

```
from sklearn.tree import DecisionTreeClassifier

# Basic decision tree classifier with common parameters
model = DecisionTreeClassifier(
    criterion='gini',
    max_depth=5,
    min_samples_split=4,
    min_samples_leaf=2,
    random_state=42
)
model.fit(X_train, y_train)
```

Refer to the scikit-learn documentation for a full list of parameters and options.

When is a Decision Tree Appropriate?

Decision trees are well-suited to certain types of problems and data:

- **When Interpretability is Important:-** Decision trees provide clear, visual decision rules that can be easily understood and explained, making them ideal for applications where model transparency is needed.
- **For Data with Mixed Feature Types:-** They naturally handle both numerical and categorical features without the need for complex encoding.
- **With Non-linear Relationships:-** Decision trees can capture complex, non-linear relationships between features and the target variable using hierarchical splits.
- **For Handling Missing Values:-** Some decision tree implementations can handle missing values internally or make splits that are robust to missing data.

- **When Minimal Data Preprocessing is Desired:** Unlike many algorithms, decision trees often require little to no feature scaling or normalization.
- **For Feature Selection:** Trees inherently perform feature selection by prioritizing the most informative features for splitting.
- **When Dealing with Interactions:** Trees can model interactions between variables without explicit feature engineering.

Naive Bayes Classifier

The **Naive Bayes** classifier is a family of simple yet effective probabilistic classification algorithms based on **Bayes' Theorem**, with the "naive" assumption that all features are independent of each other given the class label.

When to Use Naive Bayes?

- Best suited for high-dimensional data and applications involving text data (spam filtering, sentiment analysis, document classification).
- Performs well when the independence assumption roughly holds or when you need a fast, baseline classifier.

How Does Naive Bayes Work?

Naive Bayes works by applying Bayes' Theorem to calculate the probability that a given instance belongs to each possible class, using the observed feature values.

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

Where:

- P(A|B) - Posterior Probability
- P(B|A) - Likelihood
- P(A) - Prior Probability
- P(B) - Evidence/Marginal Probability

- **Posterior Probability:** The probability of the class (A) given the observed features (B). This is the value the model aims to predict.
- **Likelihood:** The probability of observing the features (B) given the class ((A)) is true. (P(A))
- **Prior Probability:** The initial probability of the class ((A)) occurring before any evidence (features) is considered.
- **Evidence/Marginal Probability:** The probability of observing the given features ((B)) across all classes. In classification, this is a constant scaling factor for all classes and can be ignored when simply comparing probabilities to make a prediction.

Types of Naive Bayes

Naive Bayes classifiers come in a few main variants, each suited to a specific kind of data:

- **Gaussian Naive Bayes**
  This type assumes that the features follow a normal (Gaussian) distribution within each class. It is used when the data are continuous and can be reasonably modeled as bell curves. For example, it works well for things like physical measurements (e.g., height, length) that tend to be normally distributed.

- **Multinomial Naive Bayes**
  Designed for discrete count data, this variant is commonly used in text classification where features represent counts, such as word frequencies in documents (e.g., how many times each word appears in an email). It assumes each feature represents the number of times an event occurs.

- **Bernoulli Naive Bayes**
  Best for binary/boolean features, where each feature only records if something is present (1) or absent (0). It is often used for tasks where the inputs are true/false ("Is this word present in the document or not?") rather than counts.

Choosing the right type depends on the nature of your features: use GaussianNB for continuous data, MultinomialNB for counts, and BernoulliNB for binary indicators.

Example Usage

```
from sklearn.naive_bayes import GaussianNB, MultinomialNB

# Example 1: Gaussian Naive Bayes for continuous data
model = GaussianNB(var_smoothing=1e-8)
model.fit(X_train, y_train)

# Example 2: Multinomial Naive Bayes for count data (e.g., text)
model = MultinomialNB(alpha=1.0, fit_prior=True)
model.fit(X_train_counts, y_train)
```

Refer to the scikit-learn documentation for a full list of classes, parameters, and options.

When not to Use Naive Bayes

Naive Bayes is not a great choice when:

- Feature dependencies are critical (e.g., time series where sequence matters)
- Features are highly correlated (violates independence assumption severely)
- Small dataset with many features (can lead to zero probabilities without smoothing)
- Need probability calibration (Naive Bayes probabilities are often poorly calibrated)

Pros and Cons

- **Advantages**
  - Simple, fast, and easy to implement.
  - Works well with small datasets and high-dimensional spaces.
  - Handles both binary and multiclass problems.
  - Requires little training data.
- **Limitations**
  - Performs poorly if features are highly correlated (violates independence assumption).
  - Output probabilities can be poorly calibrated.

K-Nearest Neighbors (KNN)

K-Nearest Neighbors (KNN) is a simple, intuitive, and versatile supervised machine learning algorithm used for both classification and regression tasks.

- **How it Works**
  KNN predicts the label (for classification) or value (for regression) of a new instance by looking at the 'k' closest examples in the training data (its “neighbors”). The output is determined by a majority vote (classification) or by averaging the values (regression) of these neighbors.

Key Characteristics of KNN

- **Instance-based learning:** KNN does not build an explicit model; instead, it memorizes the training data.
- **Distance metric:** Common default is Euclidean distance for continuous features, but others (Manhattan, Minkowski, etc.) can be used.
- **Parameter k:** Controls the number of neighbors to consider.

When to use KNN

- When you need a simple baseline for a classification or regression problem.
- When training speed is not a concern (KNN can be slow at prediction time for large datasets).
- When the relationship between features and output is non-linear and local neighborhoods are meaningful.

Example KNN for Classification in Python

```
from sklearn.neighbors import KNeighborsClassifier

# X_train: feature matrix; y_train: class labels

knn = KNeighborsClassifier(n_neighbors=5, metric='euclidean')
knn.fit(X_train, y_train)

# Predict new labels
y_pred = knn.predict(X_test)
```

Common Parameters in scikit-learn Estimators

Most scikit-learn estimators (like KNN, LinearRegression, etc.) share a common set of parameters and conventions:

- `fit(X, y)` : Trains the model using feature matrix `X` and labels/targets `y` .
- `predict(X)` : Predicts targets for new data `X` .
- `n_neighbors` : Number of neighbors (for KNN).
- `metric` : Distance function to use (for KNN; e.g., 'euclidean', 'manhattan').

You can always see all parameters and their descriptions for any estimator in the official scikit-learn documentation or by calling `help(ModelClass)` or inspecting `ModelClass().get_params()` in code.

Scenario	Recommended Metric	Why
Physical/geometric data	Euclidean	Natural for spatial distances
Text documents	Cosine	Ignores document length, measures content similarity
High dimensions, sparse data	Cosine or Manhattan	Euclidean suffers from "curse of dimensionality"
Grid-based data (images, maps)	Manhattan	Natural for grid movements
Outliers present	Manhattan	Less sensitive than Euclidean
Binary features	Hamming or Jaccard	Specialized for binary comparisons
All features equally important	Euclidean (normalized)	Balanced view
Some features more critical	Weighted Minkowski	Can assign weights

> **Tip** Before applying distance metrics, consider normalizing or standardizing your features, especially if they are measured on different scales.

Strengths

- Very simple and easy to implement.
- No assumptions about data distribution.
- Naturally handles multi-class problems.

Weaknesses

- Prediction can be slow on large datasets (needs to check all points at predict time).
- Sensitive to irrelevant features and scale of data (feature scaling is recommended).
- Poor performance with high-dimensional data (curse of dimensionality).

Ensemble Methods

Ensemble methods are powerful machine learning techniques that combine the predictions of multiple models (often called "base estimators" or "weak learners") to produce a more robust, accurate, and stable model than any single one alone.

Why Use Ensembles?

- Aggregating diverse models can reduce overfitting and variance.
- They often perform better on complex or noisy datasets.
- Examples include top Kaggle competition solutions and real-world industrial systems.

Common Types of Ensemble Methods

Method	Description	Popular Example
Bagging	Trains multiple models independently on random subsets (with replacement) of the data; averages or votes the predictions.	Random Forest
Boosting	Sequentially trains models; each tries to correct errors of the previous one.	AdaBoost, Gradient Boosting, XGBoost, LightGBM
Stacking	Combines predictions of several models (possibly of different types) using another model (meta-learner).	Sklearn `StackingClassifier` / `Regressor`

Bagging (Bootstrap Aggregating)

Bagging (Bootstrap Aggregating) is an ensemble learning technique that improves the accuracy and stability of machine learning algorithms by training multiple versions of a model on different random subsets of the original dataset (drawn with replacement).

> The predictions from these base models are then combined—using averaging for regression or majority voting for classification—to produce a final, aggregated prediction.

- This approach helps to reduce variance and overfitting, particularly for high-variance models like decision trees.
- **Common example:** Random Forest (a collection of decision trees).

Bagging with Decision Trees

```
from sklearn.ensemble import BaggingClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Load example dataset
X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42)

# Create a Bagging Classifier with Decision Trees as base estimators
bagging = BaggingClassifier(
    base_estimator=DecisionTreeClassifier(),
    n_estimators=10,       # number of trees
    random_state=42
)
bagging.fit(X_train, y_train)
y_pred = bagging.predict(X_test)

print("Bagging Accuracy:", accuracy_score(y_test, y_pred))
```

Random Forest Example in Python

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Load example dataset
X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2, random_state=42)

# Create a Random Forest with 100 trees
rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)
y_pred = rf.predict(X_test)
# Predict the class for a single new sample represented as an array
single_pred = rf.predict([[5.1, 3.5, 1.4, 0.2]])
print("Prediction for array input sample:", single_pred[0])

print("Random Forest Accuracy:", accuracy_score(y_test, y_pred))
```

Boosting

Boosting is an ensemble technique that builds a strong predictor by sequentially adding models, each new model tries to fix the errors made by the ensemble so far. Earlier models may misclassify certain data points; boosting increases their influence so that later models focus on these “harder” cases.

- Models are trained sequentially, each focusing on correcting the mistakes of the previous ones. Assigns higher weight to previously misclassified examples.
- This process reduces both bias and variance, often achieving high accuracy.
- Can create very accurate models by focusing on hard-to-predict instances.
- **Common examples:** AdaBoost, Gradient Boosting, XGBoost, CatBoost, LightGBM.

AdaBoost in Python

```
from sklearn.ensemble import AdaBoostClassifier
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Load example dataset
X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Create an AdaBoost classifier with 50 weak learners (default is DecisionTreeClassifier stumps)
ada = AdaBoostClassifier(n_estimators=50, random_state=42)
ada.fit(X_train, y_train)
y_pred = ada.predict(X_test)

print("AdaBoost Accuracy:", accuracy_score(y_test, y_pred))
```

Gradient Boosting in Python

```
from sklearn.ensemble import GradientBoostingClassifier

# Using the same data as above
gb = GradientBoostingClassifier(n_estimators=100, random_state=42)
gb.fit(X_train, y_train)
y_pred = gb.predict(X_test)

print("Gradient Boosting Accuracy:", accuracy_score(y_test, y_pred))
```

Stacking

Stacking combines multiple base models (such as decision trees, support vector machines, and logistic regression), whose predictions are then used as input (features) to train a higher-level "meta-model."

- The `meta-model` learns how to best combine the base models' predictions to improve overall performance.
- Allows combination of heterogeneous models to leverage the strengths of diverse models and reduce individual weaknesses.
- Trains multiple base models (can be different types, e.g., tree, logistic regression, SVM). Their predictions become features for a *meta-model* ("stacked" on top) that makes the final prediction.

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import StackingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score
# Load example dataset
X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Define base models
base_learners = [('dt', DecisionTreeClassifier(random_state=42)), ('svc', SVC(probability=True, random_state=42))]

# Meta-model
meta_learner = LogisticRegression()
# Create the stacking classifier

stack = StackingClassifier(estimators=base_learners, final_estimator=meta_learner, cv=5)
# Train and evaluate
stack.fit(X_train, y_train)
y_pred = stack.predict(X_test)
print("Stacking Classifier Accuracy:", accuracy_score(y_test, y_pred))
```

When to Use Stacking

- When you have multiple diverse base models whose predictions you wish to combine for potentially higher accuracy.
- When single model performance has plateaued even after tuning, and combining models is likely to reduce bias or variance.
- When your data or problem is complex and you suspect no single algorithm will capture all patterns.
- When you want to leverage both simple and complex models, mixing classical models (like logistic regression, SVM) and more complex ones (such as trees or neural nets).
- For competitions (like Kaggle) and applied projects where maximizing predictive performance is more important than interpretability or simplicity.

Regression Algorithms

Regression is a core task in supervised learning where the goal is to predict a *continuous* output variable (target) from one or more input features. Unlike classification (which predicts discrete class labels), regression predicts quantities that can take any real value.

Typical regression problems include:

- Predicting housing prices based on features like square footage, location, and number of bedrooms.
- Estimating the temperature for a given day based on weather conditions.
- Forecasting sales, stock prices, or demand.

1. Linear Regression

Linear regression is the simplest form of regression and serves as a fundamental building block for understanding more complex regression algorithms.

> The main idea is to model the relationship between one or more input features (**independent variables**) and a continuous output (**dependent variable**) by fitting a straight line (or hyperplane in higher dimensions) that best predicts the output values.

The core assumptions of linear regression are:

- The relationship between inputs and output is linear.
- The errors (residuals) are normally distributed and homoscedastic (constant variance).
- The features are not highly collinear.
- In practice, linear regression finds the best-fitting line by minimizing the sum of squared errors (the difference between actual and predicted values).
- It's quick to train, easy to interpret, and forms the baseline for many regression tasks.
- However, if the relationship in your data is highly non-linear or has interactions that are not captured by individual features, linear regression may not perform well.

```python
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error

# Load sample regression data
from sklearn.datasets import make_regression
X, y = make_regression(n_samples=200, n_features=3, noise=10, random_state=42)

# Split data into training and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Create and train the Linear Regression model
reg = LinearRegression()
reg.fit(X_train, y_train)

# Make predictions on the test set
y_pred = reg.predict(X_test)
print("Linear Regression MSE:", mean_squared_error(y_test, y_pred))
```

Polynomial Regression

Polynomial regression is an extension of linear regression where the relationship between independent variables and the dependent variable is modeled as an nth degree polynomial. It enables fitting non-linear curves to the data, letting you capture more complex relationships.

> In practice, you create additional polynomial features from the original features (for example, (x^2, x^3)), and then fit a linear regression model on these new features.

When to use Polynomial Regression

- When the relationship between the input and output appears non-linear, but can be well-approximated by a polynomial.
- When a simple linear model underfits the data.

> Higher-degree polynomials may lead to overfitting.

```
from sklearn.preprocessing import PolynomialFeatures

# Create polynomial features (e.g., degree=2 for quadratic)
poly = PolynomialFeatures(degree=2)
X_poly = poly.fit_transform(X_train)
X_test_poly = poly.transform(X_test)

# Fit linear regression on the transformed features
reg_poly = LinearRegression()
reg_poly.fit(X_poly, y_train)

# Predict on the polynomial-transformed test set
y_pred_poly = reg_poly.predict(X_test_poly)
print("Polynomial Regression MSE:", mean_squared_error(y_test, y_pred_poly))
```

2. Decision Tree Regression

A decision tree regressor predicts continuous values by recursively splitting the data into smaller groups based on feature values, forming a tree structure.

- At each internal node, the model chooses the best feature and split point to minimize prediction error (e.g., mean squared error).
- The prediction for a sample is the average target value of the training samples in the leaf node where that sample falls.
- Use when you suspect the underlying relationship between features and the target is non-linear, or when interactions between features are important.
- Decision trees also handle both numerical and categorical features natively and are easy to visualize and interpret.

When to use Decision Tree Regression

- **When the relationship between features and the target is non-linear.** Decision trees can naturally model complex, non-linear patterns that linear regression cannot.
- **When the data contains both numerical and categorical features.** Decision trees handle both types natively without the need for one-hot encoding.
- **When the data has missing values.** Decision trees can handle missing values in features without special preprocessing.
- **When interpretability is important.** Decision trees are easy to visualize and interpret, allowing insight into the decision-making process.
- **When you want relatively fast model training and prediction for small to medium-sized datasets.**

- On very small datasets or highly noisy problems, decision trees are prone to overfitting. Consider limiting the tree depth or using ensemble techniques (e.g., Random Forest, Gradient Boosting) for better generalization.
- They may not perform as well as other models when the true relationship in the data is very smooth and linear.

```
from sklearn.tree import DecisionTreeRegressor

# Create and train the Decision Tree Regressor
tree = DecisionTreeRegressor(random_state=42)
tree.fit(X_train, y_train)

# Predict on the test set
y_pred_tree = tree.predict(X_test)
print("Decision Tree Regression MSE:", mean_squared_error(y_test, y_pred_tree))
```

3. Random Forest Regression

Trains multiple decision trees on random subsets of data and features, then averages their predictions for better accuracy and reduced overfitting.

- **Use -** When you want high accuracy and can trade some interpretability for robustness.

```
from sklearn.ensemble import RandomForestRegressor

# Create and train the Random Forest Regressor
rf = RandomForestRegressor(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)

# Predict on the test set
y_pred_rf = rf.predict(X_test)
print("Random Forest Regression MSE:", mean_squared_error(y_test, y_pred_rf))
```

4. Support Vector Regression (SVR)

Tries to fit as many data points as possible between two parallel lines (the margin), allowing for some errors.

- Supports both linear and non-linear regression via kernels.
- **Used** When you want a flexible method that's robust to outliers or need to model non-linear relationships.

```
from sklearn.svm import SVR

# Create and train the Support Vector Regressor (kernel='rbf(radial basic function)' gives non-linearity)
# linear, polynomial kernel
svr = SVR(kernel='rbf')
svr.fit(X_train, y_train)

# Predict on the test set
y_pred_svr = svr.predict(X_test)
print("Support Vector Regression MSE:", mean_squared_error(y_test, y_pred_svr))
```

5. K-Nearest Neighbors Regression

Predicts values based on the average (or weighted average) target of the k most similar samples (neighbors).

- **Use -** When the relationship between features and target is hard to model parametrically, and "local similarity" makes sense.

```
from sklearn.neighbors import KNeighborsRegressor

# Create and train the KNN Regressor
knn = KNeighborsRegressor(n_neighbors=5)
knn.fit(X_train, y_train)

# Predict on the test set
y_pred_knn = knn.predict(X_test)
print("KNN Regression MSE:", mean_squared_error(y_test, y_pred_knn))
```

Strengths - Simple, no training phase (lazy learner), can model complex relationships.

Limitations - Prediction can be slow on large datasets, sensitive to feature scaling and irrelevant features.

> For many regression algorithms (especially KNN and SVR), scaling your features with `StandardScaler` or `MinMaxScaler` can improve performance. Also, hyperparameter tuning (e.g., tree depth, `n_neighbors`, kernel choices) is critical for best results.

Cross Validation

Cross-validation is a robust statistical technique used in machine learning for evaluating and validating models.

> Main goal is to assess how a predictive model will generalize to an independent (unseen) dataset, helping to prevent issues like overfitting.

Cross-validation can be applied in almost all supervised machine learning tasks, both **classification, regression** and sometimes in unsupervised learning.

> **Note:** While cross-validation is standard for supervised learning, in unsupervised learning it usually requires additional creativity or indirect validation approaches.

How Cross-Validation Works?

- **Data Splitting:** Instead of using one train-test split, cross-validation divides the original dataset into multiple subsets (called "folds").
- **Training and Evaluating:** The model is trained on a combination of these folds and tested on the remaining fold(s). This process is repeated so that every data point becomes part of a test set exactly once.
- **Performance Averaging:** The evaluation metric (e.g., accuracy, RMSE) is computed for each split, and the final reported score is the average across all folds, offering a more reliable estimate of model performance.

Why use Cross-Validation?

- Provides a better estimate of true model performance by reducing variance from a single random split.
- Helps with hyperparameter tuning, as it avoids overfitting to a particular test set.
- Useful for model selection and comparison.

K-Fold Cross-Validation

The dataset is divided into *k* equal (or nearly equal) parts, called *folds*.

- The model is trained on *k-1* folds and tested on the remaining fold.
- This process repeats *k* times, with each fold used once as the test set.
- Results are averaged over all runs.

This method helps obtain a more reliable measure of model performance by validating it on multiple train-test splits.

```python
from sklearn.datasets import load_iris
from sklearn.model_selection import KFold, cross_val_score
from sklearn.linear_model import LogisticRegression
import numpy as np
# Load a sample dataset
X, y = load_iris(return_X_y=True)
# Define the model
model = LogisticRegression(max_iter=200)

# Set up K-Fold cross-validation
kf = KFold(n_splits=5, shuffle=True, random_state=42)

# Compute cross-validated accuracy scores
scores = cross_val_score(model, X, y, cv=kf, scoring='accuracy')

print(f"K-Fold CV Accuracies: {scores}")
print(f"Mean Accuracy: {np.mean(scores):.3f} +/- {np.std(scores)*2:.3f}")
```

Stratified K-Fold Cross-Validation

Similar to regular k-fold, but ensures that the proportion of classes is (approximately) the same in each fold as in the whole dataset.

- This is crucial for imbalanced classification tasks, so that every fold is a good representative of the overall label distribution.

```python
from sklearn.datasets import load_iris
from sklearn.model_selection import StratifiedKFold, cross_val_score
from sklearn.linear_model import LogisticRegression
import numpy as np

# Load a sample dataset
X, y = load_iris(return_X_y=True)
# Set up Stratified K-Fold (maintains class distribution across folds)
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
# Define the model
model = LogisticRegression(max_iter=200)
# Compute cross-validated accuracy scores
scores = cross_val_score(model, X, y, cv=skf, scoring='accuracy')

print(f"Stratified K-Fold CV Accuracies: {scores}")
print(f"Mean Accuracy: {np.mean(scores):.3f} +/- {np.std(scores)*2:.3f}")
```

Leave-One-Out Cross-Validation (LOOCV)

This is a special case of k-fold where *k* equals the total number of data points.

- For each iteration, the model is trained on all but one data point and tested on the single left-out data point.
- This repeats for every data point.
- While it uses data very efficiently, it can be computationally expensive on large datasets.

```
from sklearn.model_selection import LeaveOneOut, cross_val_score
from sklearn.linear_model import LogisticRegression
import numpy as np

# Load the dataset
X, y = load_iris(return_X_y=True)

# Set up Leave-One-Out Cross-Validation
loo = LeaveOneOut()
model = LogisticRegression(max_iter=200)

# Compute cross-validated accuracy scores (each sample is left out once)
scores = cross_val_score(model, X, y, cv=loo, scoring='accuracy')

print(f"LOOCV Accuracies (first 10): {scores[:10]}")
print(f"Mean LOOCV Accuracy: {np.mean(scores):.3f}")
```

Holdout Validation (Train/Test Split)

The data is split once into a training set and a test set. The model is trained on the training data and evaluated on the test data. `Simple, fast, but may be sensitive to how the data is randomly split`.

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
# Split data into train and test sets (e.g., 80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
# Train the model
model = LogisticRegression(max_iter=200)
model.fit(X_train, y_train)
# Make predictions on test data
y_pred = model.predict(X_test)
# Evaluate accuracy on the holdout test set
print(f"Holdout Test Accuracy: {accuracy_score(y_test, y_pred):.3f}")
```

Unsupervised Learning Algorithms

Unsupervised Learning

Unsupervised learning is a class of machine learning techniques where models are trained using data without explicit labels. The goal is to uncover patterns, structure, or insights directly from the data. Unlike supervised learning, there is no "correct answer" provided during training.

Main Objectives of Unsupervised Learning

- Discover hidden structure in data.
- Group similar data points together (clustering).
- Reduce dimensionality while retaining essential information (dimensionality reduction).
- Learn data distributions and detect anomalies.

Applications of Unsupervised Learning

Unsupervised learning has many important real-world applications, including:

- **Clustering for Customer Segmentation**
  Businesses group customers with similar behaviors or attributes together to tailor marketing strategies or personalize recommendations.
- **Anomaly Detection**
  Detecting unusual patterns in data, useful for fraud detection in finance, network security, or medical diagnostics.
- **Dimensionality Reduction and Visualization**
  Techniques like PCA help reduce high-dimensional data to visualize it or speed up further processing.

- **Image and Document Organization**
  Automatically grouping similar images or documents to simplify searching and categorization.
- **Preprocessing and Feature Extraction**
  Extracting useful features or representations from raw data, which can be used in downstream supervised tasks.
- **Recommendation Systems**
  Uncovering patterns in user interaction data to suggest items (products, movies, etc.) without explicit ratings.
- **Genomics and Bioinformatics**
  Identifying similarities in gene expression data to discover subtypes of diseases or unknown groupings in biological datasets.

Clustering in Unsupervised Learning

Clustering is a fundamental technique in unsupervised learning used to automatically group a set of data points into clusters, such that points in the same cluster are more similar to each other than to those in other clusters.

> It allows us to discover inherent groupings and structure within unlabeled data.

The effectiveness of clustering often depends on two critical factors:

- Representation of the data (which features are used and how they are scaled)
- Choice of similarity or distance metrics.

Real-world datasets may require preprocessing, such as normalization, to ensure that all features contribute equally to the formation of clusters.

Distance Metrics in Clustering

Clustering algorithms depend on **distance metrics** (or similarity metrics) to quantify how alike or different data points are. The choice of metric can have a significant impact on the results of clustering, as it defines what "similarity" means for your data.

Euclidean Distance

Measures the "straight-line" distance between two points in space.

When to use

- Features are continuous and numeric.
- All features are on similar scales (after normalization).
- Shapes and geometric distances are meaningful, such as in image or sensor data.

Scenario: You have measurements of flower petal lengths and widths (all in centimeters) and want to group similar flowers. Because your features are continuous and on the same scale, Euclidean distance works well—it measures direct geometric closeness.

Manhattan Distance

Computes the sum of absolute differences between coordinates.

When to use:

- Data is high-dimensional or sparse (e.g., text frequency vectors).
- Outliers may be present, as it is less sensitive to large differences in single features.

Scenario: You are comparing the frequency of specific words in customer reviews (e.g., how many times "good", "bad", or "quick" appear). The data is high-dimensional and counts are often zero for most words. Manhattan distance is useful here, as it effectively handles such sparse count data.

Cosine Similarity

Calculates the cosine of the angle between two vectors, focusing on their orientation rather than magnitude.

When to use:

- Measuring similarity between text documents (e.g., with TF-IDF vectors).
- Data where magnitude (vector length) varies but direction (pattern of values) is important.

Scenario: Suppose you are organizing a digital music library and want to cluster user playlists to find groups of users with similar listening habits. Each playlist is represented as a vector of song play counts, but some users listen more overall than others. Using cosine similarity allows you to group users based on the pattern of which artists or genres they listen to most, regardless of how many songs they play in total.

Jaccard Similarity

Assesses similarity based on the intersection over union of two sets.

When to use:

- Data is categorical, binary, or consists of sets (e.g., user preferences, bag-of-words models).
- Want to ignore shared zero-entries (features absent in both compared samples).

Scenario: You want to recommend friends to users in a social network, where each user's connections are represented as a set of friends. To find users with similar friend groups, you compute the Jaccard similarity between users' friend sets—the higher the overlap, the more similar their social circles, making Jaccard similarity especially suitable for these set-based recommendations.

Tip: Regardless of the metric chosen, always preprocess your data (e.g., scaling or encoding) to ensure the metric operates as intended.

For example, K-Means is best used with Euclidean distance on scaled, continuous variables, while text clustering uses cosine similarity.

Choosing the right metric is crucial to ensure the clustering algorithm captures the meaningful structure in your data.

K-Means

K-Means is one of the most popular clustering algorithms in unsupervised machine learning.

> Partition the data into *k* distinct clusters based on feature similarity.

How it works

1. Choose the number of clusters, k.
2. **Initialize:** Randomly place *k* centroids (the centers of the clusters).
3. **Assign:** Each data point is assigned to the nearest centroid, forming k clusters.
4. **Update:** Recalculate the centroid of each cluster as the mean of all points assigned to it.
5. **Repeat:** Steps 3 and 4 until cluster assignments stop changing (convergence).

Strengths

- Simple and fast for large datasets.
- Easy to implement and interpret.

Considerations

- It's must to specify `k` in advance.
- Sensitive to the initial placement of centroids (can get stuck in local minima; multiple runs help).
- Works best when clusters are spherical, similar in size, and the features are scaled.

```python
# K-Means example using the Iris dataset
from sklearn.cluster import KMeans
from sklearn.datasets import load_iris
import numpy as np

# Load the Iris dataset
iris = load_iris()
X = iris.data

# Create KMeans instance and fit to the real data
kmeans = KMeans(n_clusters=3, random_state=42)
kmeans.fit(X)

print("Cluster centers:\n", kmeans.cluster_centers_)
print("Labels:", kmeans.labels_)
```

Hierarchical Clustering

Hierarchical clustering builds a multilevel hierarchy of clusters by either a bottom-up (agglomerative) or top-down (divisive) approach.

> The agglomerative (most common) method starts with each data point as its own cluster and merges the closest pairs iteratively.

How it works (Agglomerative approach)

1. Start with each data point as its own cluster.
2. Compute the distance (similarity) between all clusters.
3. Merge the two closest clusters.
4. Repeat steps 2 and 3 until only one cluster remains or a stopping criterion is reached.

Strengths

- No need to specify the number of clusters in advance (dendrogram helps select).
- Can capture nested cluster structures.
- Dendrograms offer interpretability.

Considerations

- Computationally more expensive than K-Means for large datasets.
- Sensitive to the choice of linkage and distance metric.

```python
# Hierarchical clustering example using the Iris dataset
from sklearn.datasets import load_iris
from sklearn.cluster import AgglomerativeClustering
import matplotlib.pyplot as plt
from scipy.cluster.hierarchy import dendrogram, linkage

# Load the Iris dataset
iris = load_iris()
X = iris.data

# Fit Agglomerative Clustering with 3 clusters
agg = AgglomerativeClustering(n_clusters=3)
labels = agg.fit_predict(X)

print("Labels:", labels)

# Optional: Plotting a dendrogram for the first 30 samples
plt.figure(figsize=(8, 4))
Z = linkage(X[:30], method='ward')
dendrogram(Z, labels=iris.target[:30])
plt.title("Hierarchical Clustering Dendrogram (first 30 samples)")
plt.xlabel("Sample Index")
plt.ylabel("Distance")
plt.show()
```

DBSCAN (Density-Based Spatial Clustering of Applications with Noise)

DBSCAN is a clustering algorithm that groups together points that are closely packed (points with many nearby neighbors), and marks points that are alone in low-density regions as outliers.

> Unlike K-Means, DBSCAN does not require specification of the number of clusters and is effective at discovering clusters of arbitrary shape.

How it works

1. Specify two parameters:
   - `eps` : Neighborhood radius.
   - `min_samples` : Minimum number of points to form a dense region.
2. For each point:
   - If at least `min_samples` points are within `eps` , a new cluster is started.
   - Points within `eps` of a core point are added to the cluster.
   - Repeat expansion until density-connected points are found.
3. Points not belonging to any cluster are labeled as outliers (noise).

Strengths

- Does not require the number of clusters to be specified.
- Can find arbitrarily shaped clusters.
- Can identify noise/outliers.

Considerations

- Performance may suffer with high-dimensional data.
- Results depend heavily on the choice of `eps` and `min_samples`.
- Not ideal when clusters vary significantly in density.

```python
# DBSCAN example using the Iris dataset
from sklearn.cluster import DBSCAN
from sklearn.datasets import load_iris
import numpy as np

# Load the Iris dataset
iris = load_iris()
X = iris.data

# Create DBSCAN instance and fit to the data
dbscan = DBSCAN(eps=0.8, min_samples=5)
labels = dbscan.fit_predict(X)

print("Labels assigned by DBSCAN:", labels)
print("Number of clusters found:", len(set(labels)) - (1 if -1 in labels else 0))
print("Number of noise points:", list(labels).count(-1))
```

Applications of Clustering

- Segmenting customers into distinct groups for marketing.
- Organizing large collections of documents or images.
- Identifying communities in networks.
- Detecting abnormal or novel patterns in data.

Clustering helps transform unstructured data into meaningful groups, laying the groundwork for further analysis or action in many real-world scenarios.

Association Rule Mining

Association rule mining is a popular unsupervised learning technique used to uncover interesting relationships, patterns, and associations among variables in large datasets.

- It is especially well-known for its application in market basket analysis, where the goal is to find associations between items frequently purchased together.

Association rule mining works by scanning transactional data (such as customer purchases) and finding patterns that describe how items or features are associated with each other. These patterns are usually expressed as `if-then` statements, known as association rules.

For example:

- *If a customer buys bread and butter, they are likely to buy milk.*

The effectiveness of association rule mining is typically measured with three key metrics:

- **Support**: How frequently do certain items or itemsets appear together in the dataset?
- **Confidence**: When certain items are present, how likely are other items to also be present?
- **Lift**: How much more likely are items to be bought together versus being bought independently?

Association rule mining is not just limited to market basket analysis; it can be applied to a variety of domains:

- **Retail**: Understanding product affinities to improve store layout, promotions, and recommendations.
- **Healthcare**: Discovering frequent combinations of symptoms or treatments.
- **Web Usage Mining**: Identifying navigation patterns among website users.
- **Bioinformatics**: Finding associations among genetic markers or symptoms.

The process typically involves:

1. Generating all possible item combinations (itemsets) in the data.
2. Identifying which of these itemsets are frequent (meet a minimum support threshold).
3. Using these frequent itemsets to generate association rules that meet minimum confidence (and potentially lift) thresholds.

This approach helps uncover meaningful relationships within large and complex datasets, supporting better decision-making and strategic planning.

Key Concepts

- **Itemset**: A collection of one or more items.
- **Support**: The proportion of transactions that contain a particular itemset. Indicates how frequently the itemset appears.
- **Confidence**: For a rule like `A → B`, confidence measures the likelihood that B is purchased when A is purchased.
- **Lift**: The ratio of the observed support for `A → B` to that expected if A and B were independent. A lift greater than 1 indicates a positive association.

Support

Support measures how frequently an itemset appears in the dataset. For a rule like `A → B`, support is the proportion (or number) of transactions in which both A and B appear together.

> *Formula*
>
> `Support(A → B) = (Number of transactions containing both A and B) / (Total number of transactions)`

- A **high support** value indicates that the itemset or rule is relevant to a large portion of your data. These are often useful for identifying widespread patterns or trends.
- A **low support** value may point to rare or niche associations. Depending on your goals, these can be interesting but are more likely to represent noise or outliers.

Confidence

Confidence indicates the reliability of the inference made by a rule. For `A → B`, it measures how often items in B appear in transactions that also contain A.

> *Formula*
>
> `Confidence(A → B) = (Number of transactions containing both A and B) / (Number of transactions containing A)`

- A **high confidence** value means that when the antecedent is present, the consequent is very likely to be present as well. This suggests a strong rule, but keep in mind that high confidence does not always mean the rule is interesting or useful.
- However, **confidence can be misleading** if the consequent is common across transactions. For example, if bread appears in nearly every basket, almost any rule predicting bread will have a high confidence, even if it's not informative.

Lift

Lift shows how much more likely the consequent (B) is purchased when the antecedent (A) is purchased, compared to when B is purchased independently. A lift greater than 1 implies a positive association between A and B.

> *Formula*
>
> `Lift(A → B) = Confidence(A → B) / Support(B)`

- **Lift > 1:-** The occurrence of the antecedent (A) increases the likelihood of the consequent (B). `positive association` .
- **Lift = 1:-** The antecedent and consequent are independent of each other. The presence of A does not affect the probability of B. `The rule is not interesting` .
- **Lift < 1:-** The antecedent (A) reduces the likelihood of the consequent (B). A and B co-occur less frequently than would be expected if they were independent, suggesting a *negative association*.

> - Higher lift values (significantly above 1) indicate stronger, more meaningful associations.
> - Rules with a lift near or below 1 are not helpful for prediction or recommendation tasks.

Apriori Algorithm

The **Apriori Algorithm** is a foundational method in association rule mining for discovering frequent itemsets and generating association rules among them.

It is particularly useful in market basket analysis, where the goal is to identify items that frequently co-occur in transactions.

> For small or moderately sized transactional datasets, Apriori offers transparent and interpretable results.

How Apriori Works?

1. **Generate Candidate Itemsets:-** Start with all items (single itemsets). In each iteration, combine frequent itemsets from the previous step to generate larger candidate itemsets (of size k+1 from size k).
2. **Prune Candidates:-** Eliminate candidate itemsets if any of their subsets are not frequent. This is based on the "apriori property": All subsets of a frequent itemset must also be frequent.
3. **Calculate Support:-** For each candidate itemset, count how many transactions contain it (its support). Discard those below the minimum support threshold.
4. **Repeat:-** Continue expanding itemsets until no candidate itemsets meet the minimum support.
5. **Generate Rules:-** Use the frequent itemsets to derive association rules that meet minimum confidence (and sometimes lift) thresholds.

Key Advantages

- Systematic and simple to implement.
- Drastically reduces the number of itemsets considered by pruning.

Limitations

- Can be computationally intensive for large datasets due to the iterative candidate generation and support counting steps.
- Not efficient when there are many frequent itemsets or long transactions.

Market Basket Analysis Example using Apriori (mlxtend)

```
import pandas as pd
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, association_rules
# Sample transaction data
dataset = [
    ['milk', 'bread', 'eggs'],
    ['milk', 'diapers', 'beer', 'eggs'],
    ['bread', 'butter'],
    ['milk', 'bread', 'butter', 'eggs'],
    ['bread', 'diapers', 'butter'],
]
# One-hot encode the data
te = TransactionEncoder()
te_ary = te.fit(dataset).transform(dataset)
df = pd.DataFrame(te_ary, columns=te.columns_)
# Find frequent itemsets
frequent_itemsets = apriori(df, min_support=0.4, use_colnames=True)
# Generate association rules
rules = association_rules(frequent_itemsets, metric="confidence", min_threshold=0.7)
print("Frequent Itemsets:\n", frequent_itemsets)
print("\nAssociation Rules:\n", rules[['antecedents', 'consequents', 'support', 'confidence', 'lift']])
```

FP-Growth Algorithm

FP-Growth (Frequent Pattern Growth) is a popular algorithm to mine frequent itemsets and generate association rules without candidate generation, making it more efficient than Apriori, especially with large datasets.

How FP-Growth Works?

1. **Build FP-Tree:** Compresses the dataset into a compact structure called the FP-Tree, which retains itemset association information.
2. **Generate Frequent Itemsets:** Recursively mines the FP-Tree by exploring conditional trees, avoiding candidate generation.

Advantages

- Highly efficient for large datasets as it reduces database scans and avoids candidate explosion.
- More scalable than Apriori for dense data.

Limitations

- FP-Tree may not fit in memory if the dataset is extremely large or lacks enough overlap among transactions.

Market Basket Analysis Example using FP-Growth (mlxtend)

```
import pandas as pd
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import fpgrowth, association_rules
# Sample transaction data
dataset = [
    ['milk', 'bread', 'eggs'],
    ['milk', 'diapers', 'beer', 'eggs'],
    ['bread', 'butter'],
    ['milk', 'bread', 'butter', 'eggs'],
    ['bread', 'diapers', 'butter'],
]
# One-hot encode the data
te = TransactionEncoder()
te_ary = te.fit(dataset).transform(dataset)
df = pd.DataFrame(te_ary, columns=te.columns_)
# Find frequent itemsets using FP-Growth
frequent_itemsets = fpgrowth(df, min_support=0.4, use_colnames=True)
# Generate association rules
rules = association_rules(frequent_itemsets, metric="confidence", min_threshold=0.7)
print("Frequent Itemsets (FP-Growth):\n", frequent_itemsets)
print("\nAssociation Rules (FP-Growth):\n", rules[['antecedents', 'consequents', 'support', 'confidence', 'lift']])
```

Applications of Association Role Mining

- Market basket analysis in retail
- Recommender systems
- Web usage mining
- Bio-informatics for gene association

Association rule mining enables organizations to uncover hidden patterns and relationships, supporting data-driven decision-making in various domains.

Dimensionality Reduction

Dimensionality reduction is a technique used in machine learning and data analysis to reduce the number of input variables (features) in a dataset while preserving as much information as possible. High-dimensional data can suffer from the "curse of dimensionality," which makes analysis and visualization more difficult and computationally expensive.

Dimensionality reduction helps in:

- Simplifying models to prevent overfitting.
- Visualizing high-dimensional data in low dimensional format.
- Reducing computational cost and storage requirements.
- Improving model performance by removing noise and redundant features.

Principal Component Analysis (PCA)

Principal Component Analysis (PCA) works by identifying the directions (principal components) along which the variance in the data is maximized.

- PCA is unsupervised, it does not use any class labels during the transformation, and is most useful with continuous, linearly related features.
- It is a powerful tool for data exploration and preprocessing in many machine learning workflows.

When to use PCA

- To remove noise and redundancy from data by keeping only the most informative components.
- When you want to speed up machine learning algorithms that struggle with very high-dimensional data.
- To mitigate overfitting, especially when the number of features is much larger than the number of observations.

Principal Component Analysis (PCA)

```
from sklearn.datasets import load_iris
from sklearn.decomposition import PCA

# Load a real dataset
iris = load_iris()
X = iris.data

# PCA for dimensionality reduction
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)

# Explained variance ratio
print(f"Explained variance ratio: {pca.explained_variance_ratio_}")
print(f"Total explained variance: {pca.explained_variance_ratio_.sum():.3f}")
```

t-Distributed Stochastic Neighbor Embedding (t-SNE)

t-Distributed Stochastic Neighbor Embedding (t-SNE) is a nonlinear technique primarily used for visualizing high-dimensional data in 2 or 3 dimensions. Unlike PCA, which is a linear method, t-SNE is particularly effective at capturing complex structures by modeling similarities among points.

- t-SNE works by minimizing the divergence between probability distributions representing pairwise similarities in high and low dimensions.
- It's widely used for data exploration, pattern discovery, and visualizing clusters in data (such as in image, text, or genomic data).
- t-SNE is unsupervised and is best suited for visualization, not for downstream quantitative modeling, as it may distort global data structure.

When to Use t-SNE

- When you want to visualize high-dimensional data in 2 or 3 dimensions, especially to explore clusters or patterns.
- When understanding the local relationships between data points is more important than global structure.
- For exploratory data analysis in tasks such as image, text, or biological data where patterns may not be linearly separable.

```
from sklearn.datasets import load_iris
from sklearn.manifold import TSNE
import matplotlib.pyplot as plt

# Load example dataset
data = load_iris()
X = data.data
y = data.target
# t-SNE for dimensionality reduction to 2D
tsne = TSNE(n_components=2, perplexity=30, random_state=42)
X_tsne = tsne.fit_transform(X)
plt.figure(figsize=(8, 6))
plt.scatter(X_tsne[:, 0], X_tsne[:, 1], c=y, cmap='viridis', s=40)
plt.title('2D visualization of Iris dataset using t-SNE')
plt.xlabel('t-SNE feature 1')
plt.ylabel('t-SNE feature 2')
plt.show()
```

Linear Discriminant Analysis (LDA)

Linear Discriminant Analysis (LDA) is a supervised dimensionality reduction technique, primarily used for classification problems. Unlike PCA, which is unsupervised and seeks components that maximize variance; LDA tries to find a feature subspace that best separates two or more classes.

- **Goal:** Maximize the ratio of *between-class variance* to *within-class variance* in any particular dataset, thereby guaranteeing maximum separability.
- **Use case:** When you want to reduce dimensionality while preserving as much of the class discriminatory information as possible.

LDA works by projecting your dataset onto a lower-dimensional space with a criterion that maintains class separability.

Key Concepts

- **Within-class scatter:** Measures how samples in the same class spread out from their respective means.
- **Between-class scatter:** Measures the distance between the means of different classes.
- LDA projects the data in such a way that between-class scatter is maximized while within-class scatter is minimized.

When to Use LDA

- Classification tasks with labeled data and multiple features.
- Datasets where classes are well-separated.
- When you need dimensionality reduction that emphasizes class discrimination.

Comparison with PCA

Aspect	PCA	LDA
Supervision	Unsupervised	Supervised
Optimization Goal	Maximize total variance	Maximize class separability
Uses labels?	No	Yes
Typical Output	Orthogonal axes (principal comps)	Linear discriminants

```
from sklearn.datasets import load_iris
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
import matplotlib.pyplot as plt
# Load example dataset
data = load_iris()
X = data.data
y = data.target
# LDA for dimensionality reduction to 2D
lda = LinearDiscriminantAnalysis(n_components=2)
X_lda = lda.fit_transform(X, y)
plt.figure(figsize=(8, 6))
scatter = plt.scatter(X_lda[:, 0], X_lda[:, 1], c=y, cmap='viridis', s=40)
plt.title('2D visualization of Iris dataset using LDA')
plt.xlabel('LDA feature 1')
plt.ylabel('LDA feature 2')
plt.legend(*scatter.legend_elements(), title="Classes")
plt.show()
```

Model Evaluation and Metrics

Model evaluation in machine learning refers to the process of assessing how well a machine learning model performs on unseen data.

- They provide insights into a model's predictive capabilities and help measure its performance across various tasks.

Why Evaluate Models?

Proper model evaluation ensures the machine learning model is performing as expected and generalizes well to new data.

- It's vital for avoiding overfitting, selecting good models, and demonstrating predictive performance.

Key Classification Metrics

Accuracy

- The ratio of correct predictions to total predictions (i.e., (TP + TN) / (TP + TN + FP + FN)).
- Use when the classes are balanced (class frequencies are similar) and the cost of false positives and false negatives is similar.

  Example: If 950 out of 1000 spam emails are detected correctly, the accuracy is 95%.

> **Weakness:** Can be misleading if your data is imbalanced (e.g., 95% one class and 5% another—a model predicting only the majority class achieves high accuracy but is not useful).

Precision

- The ratio of true positive predictions to all positive predictions (TP / (TP + FP)).

> `Precision answers` - of all the items the model claimed were positive, how many actually are?

- Use when the cost of a false positive is high.
  Example: For email spam detection, precision tells you how many emails labeled as spam were actually spam.
- **Precision** is useful in scenarios where it is more costly to label something as positive when it is actually negative. For example, in email spam detection, a false positive means that an important email is marked as spam. High precision reduces this risk.

Recall (Sensitivity, True Positive Rate)

- The ratio of true positive predictions to all items that are actually positive (TP / (TP + FN)).

  > Recall answers: of all the positive items, how many did the model find?

- Use when missing positive cases (false negatives) is expensive or dangerous.
  Example: In disease screening, recall reflects the ability to catch all patients with the disease.

- **Recall** is important in cases where missing a positive is costly or dangerous, such as disease detection. A low recall means many actual positive cases are missed by the model.

F1 Score

The harmonic mean of precision and recall (2 x (Precision x Recall) / (Precision + Recall)).

- Use when you need to balance precision and recall, especially on imbalanced datasets or when both kinds of errors are costly.
  Example: In information retrieval (e.g., search engines), F1 score helps balance finding all relevant documents (recall) with not flooding the results with irrelevant ones (precision).
- **F1 Score** balances both precision and recall. It is especially effective on imbalanced datasets, where relying on accuracy can be misleading.

Confusion Matrix

A table showing the counts for true positives, false positives, true negatives, and false negatives.

- Use for a comprehensive, interpretable breakdown of model performance, especially to identify which types of errors are common.
  Example: In multi-class classification, the confusion matrix can show whether certain classes are confused for others.
- **Limitations:** Not a single-number metric; needs interpretation and is less useful for directly comparing models.
- **Confusion Matrix** provides a detailed breakdown, how many actual positives and negatives were correctly or incorrectly predicted. This helps understand the types of errors the model makes.

		Predicted Class: Positive	Predicted Class: Negative	
Actual Class	Positive	True Positive (TP)	False Negative (FN) **Type II Error**	**Sensitivity** $\frac{TP}{(TP+FN)}$
	Negative	False Positive (FP) **Type I Error**	True Negative (TN)	**Specificity** $\frac{TN}{(TN+FP)}$
		Precision $\frac{TP}{(TP+FP)}$	**Negative Predictive Value** $\frac{TN}{(TN+FN)}$	**Accuracy** $\frac{TP+TN}{(TP+TN+FP+FN)}$

ROC Curve & AUC (Receiver Operating Characteristic & Area Under the Curve)

- ROC curve plots the true positive rate vs. false positive rate at various thresholds; AUC quantifies the model's ability to distinguish classes (AUC of 1 is perfect, 0.5 is random guessing).
- Use to evaluate models in binary classification, especially under class imbalance or when you need to set a specific threshold. AUC is good for comparing models regardless of threshold.
  Example: In credit card fraud detection, ROC/AUC helps assess how well the model separates fraudulent from legitimate cases.
- **Limitations:** Not as interpretable for multi-class problems; can be over-optimistic for heavily imbalanced data.

Choosing the right metric is crucial:

- Use **accuracy** if classes are balanced and all errors are equally costly.
- Use **precision** to minimize false alarms (false positives).
- Use **recall** to catch as many real cases as possible (minimize false negatives).
- Use **F1** if you need a balance between precision and recall and your data is imbalanced.
- Use the **confusion matrix** for diagnostic insight into your model's error types.
- Use **ROC/AUC** for binary (or one-vs-all multi-class) classification with imbalanced data or when model discrimination across thresholds is important.

Example (sklearn)

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, classification_report, confusion_matrix

y_true = [...]  # True labels
y_pred = [...]  # Model predictions

print("Accuracy:", accuracy_score(y_true, y_pred))
print("Precision:", precision_score(y_true, y_pred, average='weighted'))
print("Recall:", recall_score(y_true, y_pred, average='weighted'))
print("F1 Score:", f1_score(y_true, y_pred, average='weighted'))
print("Confusion Matrix:\n", confusion_matrix(y_true, y_pred))
print(classification_report(y_true, y_pred))
```

Regression Metrics

Mean Absolute Error (MAE)

Measures the average magnitude of errors in predictions, without considering direction. Lower MAE means better model performance.

- **When to use:** Prefer MAE when all individual differences are equally important and you want a metric that is robust to outliers.

Mean Squared Error (MSE)

The mean of the squared differences between predicted and actual values. Larger errors are penalized more.

- **When to use:** Use MSE when large errors are especially undesirable and should be "punished" more, e.g., in applications where big mistakes are costly.

Root Mean Squared Error (RMSE)

The square root of MSE, making it easier to interpret as it is in the same units as the target variable.

- **When to use:** Use RMSE for the same reasons as MSE, but when you also want interpretability in the original units of measurement.

R-squared (R^2)

Represents the proportion of variance in the target explained by the model. Ranges from 0 (no explanatory power) to 1 (perfect fit).

- **When to use:** Use R^2 for a quick assessment of model explanatory power, especially when comparing several models or looking for overall "fit."

Summary Table:

Metric	Use When	Notes
MAE	Want average error, robust to outliers	Equal weighting of all errors
MSE	Need to penalize large errors more	Sensitive to outliers
RMSE	Like MSE but want metric in original units	Sensitive to outliers, interpretable units
R^2	Want to assess explained variance	Not an error metric, but fit quality

Example

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

y_true = [...]  # True target values
y_pred = [...]  # Model predictions

print("MAE:", mean_absolute_error(y_true, y_pred))
print("MSE:", mean_squared_error(y_true, y_pred))
print("RMSE:", np.sqrt(mean_squared_error(y_true, y_pred)))
print("R-squared:", r2_score(y_true, y_pred))
```

Key Unsupervised Learning Metrics

- **Silhouette Score:** Measures how similar an object is to its own cluster compared to other clusters (ranges from -1 to 1; higher is better).
- **Davies-Bouldin Index:** Average similarity ratio of each cluster with its most similar cluster (lower values indicate better clustering).
- **Calinski-Harabasz Index (Variance Ratio Criterion):** Ratio of between-cluster dispersion to within-cluster dispersion (higher is better).
- **Inertia (Within-Cluster Sum-of-Squares):** Sum of squared distances from each sample to its assigned cluster center (lower indicates tighter clusters).

Example (sklearn)

```
from sklearn.metrics import silhouette_score, davies_bouldin_score, calinski_harabasz_score

X = [...]       # Feature data
labels = [...]  # Cluster labels predicted (e.g., from KMeans)

print("Silhouette Score:", silhouette_score(X, labels))
print("Davies-Bouldin Index:", davies_bouldin_score(X, labels))
print("Calinski-Harabasz Index:", calinski_harabasz_score(X, labels))
```

Complete Machine Learning Pipeline

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report
# 1. Load and prepare data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
# 2. Feature scaling
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
# 3. Train model
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train_scaled, y_train)
# 4. Make predictions
y_pred = model.predict(X_test_scaled)
# 5. Evaluate model
print(classification_report(y_test, y_pred))
```

Lab Exercise: Supervised and Unsupervised Learning

Objective: Implement and compare different machine learning algorithms

Tasks:

1. Load a dataset and perform EDA
2. Implement classification algorithms (KNN, Decision Tree, Random Forest)
3. Apply cross-validation and compare performance
4. Perform clustering analysis
5. Conduct feature selection and engineering
6. Create a complete ML pipeline

Reflection Question

- After learning about the complete machine learning pipeline and various model evaluation techniques, what do you think is the most critical step in ensuring the success of a machine learning project, and why? Support your answer with insights or examples from this chapter.

Questions

1. What are the main differences between supervised and unsupervised learning?
2. Name some examples of supervised learning algorithms.
3. Name some examples of unsupervised learning algorithms.
4. Why is data preprocessing important in a machine learning pipeline?
5. How do you evaluate the performance of a machine learning model?
6. Explain the purpose of splitting data into training and testing sets.
7. What is feature scaling, and when should it be applied?
8. In what situations might you use clustering algorithms?


THANK
YOU!